



Secure Software Engineering - CSEC3352

Secure Student Information System

Submitted by:

Dena Aldossary 202300112

Tamar Faraj 202301258

Instructor: Dr. Maen Mohammad Mustafa Hammad

Section: 201

Introduction

Data collected from the Online Secure Student Information System is a web system that maintains and safeguards students' academic grades. The system enables students to register/sign in securely and view their grades. The instructor updates grades for those courses. Since the system stores highly sensitive academic information, security is also an essential part of the software development process.

The main objective of this project is to design a secure software system through security principles and practices throughout the Software Development Life Cycle (SDLC). The project aims to protect user accounts, prevent unauthorized access, and ensure the confidentiality and integrity of student records.

System Scope, which includes:

- Account creation and updates.
- Secure login process.
- Viewing grades.
- Posting grades.
- Updating grades.
- Input validation and attack prevention.
- Student/instructor access control.

The purpose of the project is:

- Develop a secure and reliable student information system.
- Ensure the protection of student-sensitive data from cyber-attacks.
- Prevent common security vulnerabilities, especially SQL injection.
- Apply secure coding practices.
- Implement authentication and access control mechanisms.
- Test the system against potential security threats.

Security Requirements Analysis

Functional requirements:

Functional requirements describe what the system must do and the services or features it must provide to users. These requirements capture the major operations and functions of the system.

- Allow students and instructors to create accounts.
- Validate usernames and passwords before authentication.
- Display error messages for invalid login attempts.
- Allow users to update account information.
- Allow users to log in securely.
- Allow instructors to post grades.
- Allow instructors to update grades.
- Allow students to view their grades.

Nonfunctional Requirements:

Non-functional requirements specify how the system is supposed to behave. They are about system quality, performance, reliability, usability, and security rather than specific system features.

- The system must be secure and protect users' information.
- The system must be available at all times for authorized users.
- The system must have fast response times.
- The system must validate all user inputs.
- The system must prevent unauthorized access.
- The system must maintain the confidentiality and integrity of grades.
- The system must be easy to use for students and instructors.

Security Requirements:

- Student IDs must contain numbers only.
- Student IDs must be exactly 9 digits.

- Passwords must contain between 8 and 20 characters.
- The system must detect possible SQL injection attempts.
- Only authorized users can access the system.
- Students can only view their own grades.
- Instructors can only post and update grades.
- Invalid login attempts must display secure error messages.

Identify Assets:

- **Student Data:** Personal information and unique Student IDs.
- **Grade Records:** The integrity and confidentiality of academic results.
- **System Credentials:** Student IDs and passwords used for authentication.
- **Availability:** The uptime of the portal during high-traffic periods (e.g., finals week).

Identify Threats:

- **SQL Injection:** Attempts to bypass authentication using malicious input
- **Unauthorized Access:** Attempting to access the system without valid credentials
- **Invalid Input:** Entering incorrect or unexpected input values
- **Weak Passwords:** Using short or insecure passwords
- **Spoofing:** Pretending to be another user to gain access

Risk Assessment:

Threat	Likelihood	Impact	Risk Level	Mitigation
SQL Injection	High	High	Critical	Input validation and filtering
Unauthorized Access	Medium	High	High	Authentication and access control

Invalid Input Attack	High	Medium	High	Input validation
Weak Passwords	Medium	Medium	Medium	Password validation

Identify Misuse Cases:

1. A student attempts to inject SQL commands into the login field to bypass authentication.
2. A student attempts to access a URL intended for instructors (e.g., /update-grades) by guessing the path.
3. An unauthorized script attempts to automate thousands of login attempts to crack a password.

Securing Design

The Online Secure Student Information System was created based on secure software engineering principles to ensure the confidentiality, integrity, and availability of students' academic information. Such security controls were built into the overall design of the system to mitigate vulnerabilities and better avoid common cyberattacks.

System Architecture and Components:

The Online Secure Student Information System is built on a three-layer architecture to enhance security, scalability, and maintainability. The system follows a three-tier secure architecture:

1. Presentation Tier

The first layer is the presentation layer, which reflects the web browser that a student or instructor uses to interact with the system. This layer comprises the login page, account management interface, student dashboard, instructor dashboard, and grade viewing and management pages. Security controls at this layer include input validation and restricted access to authenticated users only.

2. Logic Tier

The second layer is the logic layer (or application layer), which processes user requests and enforces system logic. It contains the authentication module, authorization and role management system, grade management functions, logging system, and error handling mechanisms. This layer ensures that all user inputs are validated on the server side, regardless of any testing on the client side. The design supports Role-Based Access Control (RBAC), where students can view grades while instructors can manage grades.

3. Data Tier

The third layer is the database layer, which contains all sensitive information, student records, grades, user credentials, and system logs. In this layer, protection is ensured by restricted database permissions, separation of database roles, secure storage practices, and regular backups. Sensitive data is secured via controlled access; upcoming enhancements comprise encryption and password hashing to enrich database security.

Threat Modeling (STRIDE)

The STRIDE model was used to recognize potential security threats in the system. Security threats from spoofing are decreased through login validation and authentication verification. This is done to mitigate tampering threats via input validation and limiting access to the login system. Information disclosure risks are mitigated by preventing unauthorized users from accessing the system. Input filtering techniques are used for detecting and rejecting basic SQL injection attempts. Such security features enhance the general protection of the student information system.

Security Design Principles Applied

The Online Secure Student Information System development was based upon several fundamental principles in security design to make sure that it can withstand modern-day cyber risks. Instead of using a single line of defence, the system follows a proactive and multi-layered security plan that integrates security into the software's logic and data-handling tiers. Adopting industry-standard frameworks, the design emphasizes reducing the attack surface and ensuring that every user interaction is strictly governed by verified identity and authorized permissions.

The principle of least privilege guides the system, restricting permissions given to a user role. Students can only see their own grades; instructors can register their grades and update them. This minimizes the likelihood of unauthorized access or accidental modification of data.

Defense in depth is also applied; that is, multiple layers of security are enforced to keep the system safe. These include input validation, authentication, authorization, SQL injection prevention, and secure session handling to protect the system by implementing multiple layers of controls that secure the system. In the event one security layer fails, additional layers still exist to defend the system.

Secure authentication is maintained by validating student IDs and passwords during login to prevent unauthorized access by authenticating the student. The system verifies that student IDs follow a strict numeric format and that passwords meet length requirements. Suspicious input patterns are also detected to prevent basic injection attacks.

RBAC (Role-Based Access Control) is applied through role separation to separate the rights of different users. Students are limited to only viewing grades, and instructors can do grade management. The system itself automatically blocks unauthorized actions.

Database Security. (things we studied)

We have developed a database with strict security protections to safeguard sensitive academic material. In order to restrict access to the database, role separation and permission controls are established. Only authorized application components have the right to use the database.

The system saves the login information for students via authentication. Input validation has been performed before input processing to minimize the risk of SQL injection and invalid input attacks. Possible future enhancements include password hashing, encrypted storage, prepared statements, and stronger database access controls.

Encryption Techniques

To enhance the security of user sessions, the system will be improved by using HTTPS (TLS) communication along with password hashing and secure session tokens.

Website Security Measures.

All user inputs are validated before being processed by the system. Generic error messages on failed login attempts are given to avoid leaking confidential information about the system. SQL injection attempts are detected using input filtering techniques.

Preventive Measures Against Regular Attacks.

Input validation, along with character filtering, minimizes SQL injection attempts. The student IDs are validated to ensure they contain only numbers and are exactly 9 digits long. Password length validation is also implemented for secure login.

Secure Implementation

This online secure software system allows students to log in using their student ID and password to view their grades. This system has a log of students' usernames and passwords. When the user inputs the correct username and password, they are allowed to log in to the system. However, if the user enters the incorrect username or password, their access will be denied with an error message stating the reason. If the user tries to attempt an SQL injection attack, the system will deny their entry.

The security features implemented in this code:

- Student ID Validation:
 1. Ensures the user inputs numbers only.
 2. Only accepts a length of 9 digits.
 3. Rejects any special characters and strings.
- Password Validation:
 1. Rejects a password that has less than 8 characters.
 2. Rejects passwords that are longer than 20 characters.

- SQL Injection Prevention:
 1. Refuses entry for any user that inputs “ ‘ “ or “ = “ in the username.
 2. Refuses entry for any user that inputs “ ‘ “ or “ = “ in the password.
- Defensive Programming:
 1. The system validates the username and password before granting entry.
 2. Provides error messages when needed.

The implemented code:

```
public class SecureProject {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        String username;
        String password;
        while (true) {
            System.out.print(s: "Enter student ID: ");
            username = scan.nextLine();
            if (!username.matches(regex: "[0-9]+")) {
                System.out.println(x: "Error: Student ID must only contain numbers");
                continue;
            }
            if (username.length() != 9) {
                System.out.println(x: "Error: Student ID must be 9 digits!");
                continue;
            }
            break;
        }
        while (true) {
            System.out.print(s: "Enter password: ");
            password = scan.nextLine();
            if (password.length() < 8) {
                System.out.println(x: "Error: Password is too short.");
                continue;
            }
            if (password.length() > 20) {
                System.out.println(x: "Error: Password is too long.");
                continue;
            }
            break;
        }
    }
}
```

```

while (true) {
    System.out.print(s: "Enter password: ");
    password = scan.nextLine();
    if (password.length() < 8) {
        System.out.println(x: "Error: Password is too short.");
        continue;
    }
    if (password.length() > 20) {
        System.out.println(x: "Error: Password is too long.");
        continue;
    }
    break;
}
if (username.contains(s: "'") ||
    username.contains(s: "=") ||
    password.contains(s: "'") ||
    password.contains(s: "=")) {
    System.out.println(x: "Error: Possible SQL injection detected.");
    return;
}
if (
    (username.equals(anObject: "202300112") && password.equals(anObject: "12345678!")) ||
    (username.equals(anObject: "202301258") && password.equals(anObject: "clouds@22")) ||
    (username.equals(anObject: "202201658") && password.equals(anObject: "cherry50")) ||
    (username.equals(anObject: "202478939") && password.equals(anObject: "password!")) ||
    (username.equals(anObject: "202308356") && password.equals(anObject: "pass2424"))
) {
    System.out.println(x: "Login successful!");
} else {
    System.out.println(x: "Error: Invalid username or password.");
}
scan.close();
}
}

```

Secure Testing

Test Case 1: SQL Injection Prevention

```

| --- exec:3.1.0:exec (default-cli) @ SecureProject ---
Enter student ID: ' OR '1'='1
Error: Student ID must only contain numbers
Enter student ID: 202300112
Enter password: ' OR '1'='1
Error: Possible SQL injection detected.
-----
BUILD SUCCESS
-----
Total time: 38.838 s
Finished at: 2026-05-09T14:14:16+03:00
-----

```

When the user attempts the SQL injection attack on the login, it is rejected since it only accepts integers. Additionally, when the SQL injection attack is attempted during the password, the system responds with “Error: Possible SQL injection detected.”

Test Case 2: Length Validation

```
| --- exec:3.1.0:exec (default-cli) @ SecureProject ---  
Enter student ID: 2002  
Error: Student ID must be 9 digits!  
Enter student ID: 2012000000000000000  
Error: Student ID must be 9 digits!  
Enter student ID: 202301258  
Enter password: hello  
Error: Password is too short.  
Enter password: helloooooooooooooo123123123  
Error: Password is too long.  
Enter password: hello123123  
Error: Invalid username or password.  
-----  
BUILD SUCCESS  
-----  
Total time: 02:48 min  
Finished at: 2026-05-09T14:29:20+03:00
```

This shows that the system only accepts 9 digits for the login and rejects any passwords that aren't between 8 and 20 characters.

Test Case 3: Type Validation

```
| --- exec:3.1.0:exec (default-cli) @ SecureProject ---  
Enter student ID: Student22  
Error: Student ID must only contain numbers  
Enter student ID: 20210076!  
Error: Student ID must only contain numbers  
Enter student ID: 202300112  
Enter password: Student@pmu2  
Error: Invalid username or password.  
-----  
BUILD SUCCESS  
-----  
Total time: 02:24 min  
Finished at: 2026-05-09T14:33:31+03:00
```

This shows that the system only accepts integers for the login, but accepts integers, letters, and special characters in the password.

Test Case 4: Successful Login

```
--- exec:3.1.0:exec (default-cli) @ SecureProject ---
Enter student ID: 202201658
Enter password: cherry50
Login successful!
-----
BUILD SUCCESS
-----

Total time:  21.595 s
Finished at: 2026-05-09T14:35:41+03:00

--- exec:3.1.0:exec (default-cli) @ SecureProject ---
Enter student ID: 202301258
Enter password: clouds@22
Login successful!
-----
BUILD SUCCESS
-----

Total time:  12.445 s
Finished at: 2026-05-09T14:37:11+03:00
```

This shows that when the student enters the correct ID and password, they are granted access to the system.

Conclusion

This project showcases the design and implementation of a secure student information system with respect to secure software engineering principles. These are security measures applied during development, such as input validation, SQL injection prevention, password validation, and access control. Threat modeling, risk assessment, secure coding practices, and secure testing were some strategies in the project to identify and mitigate potential vulnerabilities.

The implemented login system validates student IDs and passwords while preventing invalid input and basic SQL injection attempts. To wrap up, the project highlights the importance of integrating security throughout the SDLC to build secure and reliable software systems.